

1. Dataset y Preprocesamiento

¿Por qué es necesario redimensionar las imágenes a un tamaño fijo para una MLP?

Una MLP recibe un vector de entrada de dimensión fija. Cada píxel de la imagen se convierte en una feature, por lo que si las imágenes tienen distintos tamaños, los vectores resultantes tendrían distinta longitud y no podrían pasar por la misma capa Linear. Se elige un tamaño fijo (por ejemplo 64×64) y todas las imágenes se escalan a ese tamaño antes de entrar a la red.

¿Qué ventajas ofrece Albumentations frente a otras librerías de transformación como *torchvision.transforms*?

Albumentations está optimizado para velocidad (usa OpenCV internamente) y ofrece un catálogo de aumentaciones mucho más amplio que *torchvision.transforms*. Además aplica la misma transformación espacial de forma consistente sobre la imagen y su máscara/keypoints en simultáneo, lo cual es clave en segmentación o detección. Para clasificación la diferencia principal es la mayor variedad de aumentaciones y la performance.

¿Qué hace `A.Normalize()`? ¿Por qué es importante antes de entrenar una red?

`A.Normalize()` estandariza los valores de los píxeles restando la media y dividiendo por la desviación estándar canal a canal (por defecto usa los valores de ImageNet: `mean=[0.485, 0.456, 0.406]`, `std=[0.229, 0.224, 0.225]`). Es importante porque las redes neuronales convergen mucho más rápido y de forma más estable cuando las entradas están en una escala similar (cercana a media 0 y varianza 1). Sin normalizar, los gradientes pueden ser muy distintos entre canales y el entrenamiento se vuelve inestable.

¿Por qué convertimos las imágenes a `ToTensorV2()` al final de la pipeline?

PyTorch opera sobre tensores, no sobre arrays de NumPy. `ToTensorV2()` convierte el array HxWxC (alto × ancho × canales) en un tensor CxHxW (canales × alto × ancho), que es el formato esperado por las capas de PyTorch. Se aplica al final para que las transformaciones anteriores (que trabajan sobre NumPy/OpenCV) ya estén completas.

2. Arquitectura del Modelo

¿Por qué usamos una red MLP en lugar de una CNN aquí? ¿Qué limitaciones tiene?

El MLP se usa como punto de partida simple para establecer un baseline. Sus limitaciones principales son: no explota la estructura espacial de las imágenes (trata cada píxel independientemente), tiene muchísimos parámetros porque la primera capa es $\text{Linear}(H \times W \times C, N)$, es sensible a traslaciones/rotaciones (no tiene invarianza espacial) y tiende a sobreajustar con datasets pequeños.

¿Qué hace la capa Flatten() al principio de la red?

Convierte el tensor de entrada de forma $C \times H \times W$ en un vector 1D de longitud $C \times H \times W$. Es necesaria porque `nn.Linear` espera un vector, no un tensor tridimensional.

¿Qué función de activación se usó? ¿Por qué no usamos Sigmoid o Tanh?

Se usa ReLU (`nn.ReLU()`). Se prefiere sobre Sigmoid y Tanh por dos razones principales: (1) no sufre del problema de *vanishing gradients* tan severamente, ya que su derivada es 1 para valores positivos; (2) es computacionalmente más barata. Sigmoid y Tanh saturan en sus extremos y sus gradientes se vuelven casi cero, lo que frena el aprendizaje en redes profundas.

¿Qué parámetro del modelo deberíamos cambiar si aumentamos el tamaño de entrada de la imagen?

El argumento `in_features` de la primera capa `nn.Linear`. Si la imagen pasa de $64 \times 64 \times 3$ a $128 \times 128 \times 3$, `in_features` debe cambiar de $64 \times 64 \times 3 = 12288$ a $128 \times 128 \times 3 = 49152$.

3. Entrenamiento y Optimización

¿Qué hace `optimizer.zero_grad()`?

Pone a cero los gradientes acumulados en los parámetros del modelo. PyTorch acumula gradientes por defecto (los suma en cada `.backward()`), por lo que si no se limpian al inicio de cada batch, los gradientes de iteraciones anteriores contaminan la actualización actual.

¿Por qué usamos `CrossEntropyLoss()` en este caso?

Es un problema de clasificación multiclase. `CrossEntropyLoss` combina internamente un `LogSoftmax` y un `NLLLoss`, lo que es numéricamente estable y apropiado cuando la salida de la red son logits (valores sin activación). Para clasificación binaria se usaría `BCEWithLogitsLoss`, y para regresión `MSELoss`.

¿Cómo afecta la elección del tamaño de batch (`batch_size`) al entrenamiento?

Batches más grandes dan estimaciones del gradiente más precisas pero requieren más memoria y cada paso de actualización es más lento. Batches más pequeños agregan ruido al gradiente, lo cual puede tener efecto regularizador y ayudar a escapar de mínimos locales, pero hacen el entrenamiento más ruidoso. En la práctica, valores entre 32 y 256 suelen funcionar bien. Con Colab y GPU, batches de 64 o 128 pueden funcionar bien también.

¿Qué pasaría si no usamos `model.eval()` durante la validación?

Las capas Dropout seguirían desactivando neuronas aleatoriamente (introduciendo ruido en la predicción) y BatchNorm usaría las estadísticas del batch actual en lugar de las estadísticas acumuladas durante el entrenamiento. Ambas cosas harían que las métricas de validación fueran incorrectas y no representativas del comportamiento real del modelo.

4. Validación y Evaluación

¿Qué significa una accuracy del 70% en validación pero 90% en entrenamiento?

Es una señal de **overfitting**: el modelo aprendió demasiado bien los datos de entrenamiento (incluyendo ruido o particularidades del conjunto) y no generaliza bien a datos nuevos. La brecha de 20 puntos indica que hay que aplicar regularización (Dropout, weight decay, data augmentation, early stopping) o conseguir más datos.

¿Qué otras métricas podrían ser más relevantes que accuracy en un problema real?

En datasets desbalanceados (pocas imágenes de una clase vs. muchas de otra), la accuracy puede ser engañosa. Son más informativas: **F1-score** (balance entre precision y recall), **AUC-ROC**, **recall por clase** (especialmente importante si el costo de falsos negativos es alto, como en detección de lesiones malignas) y la **matriz de confusión completa**.

¿Qué información útil nos da una matriz de confusión que no nos da la accuracy?

Muestra exactamente qué clases se confunden entre sí. Por ejemplo, puede revelar que el modelo casi nunca predice una clase minoritaria, o que confunde sistemáticamente dos clases parecidas. La accuracy solo dice "cuántos acerté en total" pero no dice *cuáles* ni *por qué* se equivoca.

En el reporte de clasificación, ¿qué representan precision, recall y f1-score?

- **Precision**: de todos los que el modelo predijo como clase X, ¿cuántos realmente eran clase X? (evita falsos positivos)

- **Recall:** de todos los que realmente son clase X, ¿cuántos detectó el modelo? (evita falsos negativos)
- **F1-score:** media armónica entre precision y recall. Es útil cuando hay desbalance de clases porque penaliza si alguna de las dos es muy baja.

5. TensorBoard y Logging

¿Qué ventajas tiene usar TensorBoard durante el entrenamiento?

Permite monitorear en tiempo real la evolución de la loss y las métricas en train/validación, detectar overfitting o problemas de convergencia a medida que ocurren, comparar múltiples experimentos en el mismo gráfico, visualizar imágenes de validación y distribuciones de pesos, todo sin tener que esperar a que termine el entrenamiento.

¿Qué diferencias hay entre loguear `add_scalar`, `add_image` y `add_text`?

- `add_scalar`: registra un valor numérico (loss, accuracy, lr) en función del step. Se visualiza como un gráfico de líneas.
- `add_image`: guarda un tensor de imagen para visualizar ejemplos del dataset o predicciones. Útil para ver si el modelo falla en casos específicos.
- `add_text`: guarda texto plano (configuración del experimento, hiperparámetros, notas). Sirve para documentar cada run.

¿Por qué es útil guardar visualmente las imágenes de validación en TensorBoard?

Permite ver de forma cualitativa qué imágenes clasifica mal el modelo, detectar patrones sistemáticos en los errores (por ejemplo, confunde siempre imágenes oscuras o con cierto patrón) y verificar que el preprocesamiento y la normalización se están aplicando correctamente.

¿Cómo se puede comparar el desempeño de distintos experimentos en TensorBoard?

Guardando cada experimento en una subcarpeta distinta dentro del directorio de logs (por ejemplo `runs/exp_lr001_bs64/`). TensorBoard carga automáticamente todas las subcarpetas y superpone las curvas en el mismo gráfico, permitiendo comparar directamente loss y métricas entre experimentos.

6. Generalización y Transferencia

¿Qué cambios habría que hacer si quisiéramos aplicar este mismo modelo a un dataset con 100 clases?

Solo cambiar el argumento `num_classes` (o el `out_features` de la última capa Linear) de la cantidad actual a 100. El resto del pipeline (loss, softmax implícito en `CrossEntropyLoss`, métricas) funciona igual para cualquier número de clases.

¿Por qué una CNN suele ser más adecuada que una MLP para clasificación de imágenes?

Las CNNs explotan la estructura espacial de las imágenes mediante convoluciones locales que detectan bordes, texturas y patrones jerárquicos. Son invariantes a traslaciones (por el pooling) y tienen muchos menos parámetros que un MLP equivalente porque comparten pesos en los filtros. Un MLP trata cada píxel como independiente y pierde toda la información espacial.

¿Qué problema podríamos tener si entrenamos este modelo con muy pocas imágenes por clase?

Overfitting severo: el modelo memoriza los pocos ejemplos disponibles en lugar de aprender características generalizables. Además, las estimaciones de métricas (accuracy, F1) son poco confiables con pocos datos de validación. Las soluciones típicas son data augmentation agresiva, transfer learning, o regularización fuerte.

¿Cómo podríamos adaptar este pipeline para imágenes en escala de grises?

Cambiar el número de canales de entrada de 3 a 1. En la normalización, usar mean y std con un solo valor. En `in_features` de la primera capa Linear, usar $H \times W \times 1$ en lugar de $H \times W \times 3$. Si se usa un modelo preentrenado en ImageNet (RGB), se puede replicar el canal gris 3 veces o reemplazar la primera capa conv.

7. Regularización

Preguntas teóricas

¿Qué es la regularización en el contexto del entrenamiento de redes neuronales?

Conjunto de técnicas que reducen el overfitting penalizando la complejidad del modelo o introduciendo ruido durante el entrenamiento, con el objetivo de mejorar la generalización a datos no vistos.

¿Cuál es la diferencia entre Dropout y regularización L2 (weight decay)?

Dropout desactiva aleatoriamente una fracción de neuronas en cada forward pass durante el entrenamiento, forzando a la red a no depender de neuronas específicas. L2 (weight decay) agrega un término de penalización al loss proporcional a la magnitud de los pesos, lo que los mantiene pequeños y evita que la red aproveche features ruidosas. Actúan por mecanismos distintos y son complementarios.

¿Qué es BatchNorm y cómo ayuda a estabilizar el entrenamiento?

BatchNorm normaliza la salida de cada capa para que tenga media 0 y varianza 1 dentro de cada mini-batch, y luego aplica parámetros aprendibles (gamma y beta) para escalar y desplazar. Reduce el problema de *internal covariate shift* (el cambio de distribución de activaciones durante el entrenamiento), lo que permite usar learning rates más altos y hace el entrenamiento más estable.

¿Cómo se relaciona BatchNorm con la velocidad de convergencia?

Al estabilizar las distribuciones de activación, BatchNorm permite usar learning rates más grandes sin que el entrenamiento diverja, lo que se traduce directamente en convergencia más rápida (menos épocas para alcanzar el mismo nivel de loss).

¿Puede BatchNorm actuar como regularizador? ¿Por qué?

Sí, en cierta medida. Introduce ruido en las activaciones porque las estadísticas (media y varianza) se calculan sobre el mini-batch actual, que es una estimación ruidosa de las estadísticas globales. Este ruido tiene un efecto regularizador similar (aunque menor) al de Dropout. Por eso, al usar BatchNorm suele reducirse la cantidad de Dropout necesaria.

¿Qué efectos visuales podrías observar en TensorBoard si hay overfitting?

La loss de entrenamiento sigue bajando mientras la loss de validación se estanca o sube. La accuracy de entrenamiento sigue subiendo mientras que la de validación se estanca o baja. La brecha entre ambas curvas crece con el tiempo.

¿Cómo ayuda la regularización a mejorar la generalización del modelo?

Evita que el modelo aprenda a memorizar los datos de entrenamiento (incluyendo ruido) y lo fuerza a aprender patrones más generales que también aparecen en datos nuevos. Reduce la varianza del modelo a costa de un pequeño aumento de bias, lo cual es un trade-off favorable cuando los datos son limitados.

Preguntas prácticas (resultados esperados al ejecutar)

¿Qué efecto tuvo BatchNorm en la estabilidad y velocidad del entrenamiento?

Se espera que con BatchNorm la loss baje de forma más suave y rápida desde las primeras épocas, con menos oscilaciones. La convergencia debería ser notablemente más rápida comparada con el modelo sin BatchNorm.

¿Cambió la performance de validación al combinar BatchNorm con Dropout?

La combinación suele dar mejor accuracy de validación que cualquiera de los dos por separado, ya que atacan el overfitting por mecanismos complementarios. Sin embargo, Dropout con p muy alto (>0.5) puede interferir con BatchNorm si los batch sizes son pequeños.

¿Qué combinación de regularizadores dio mejores resultados en tus pruebas?

(Falta completar con los datos reales)

¿Notaste cambios en la loss de entrenamiento al usar BatchNorm?

Sí: la loss de entrenamiento suele descender más rápido y de forma más suave. En TensorBoard se ve como una curva menos ruidosa y con mayor pendiente en las primeras épocas.

8. Inicialización de Parámetros

Preguntas teóricas

¿Por qué es importante la inicialización de los pesos en una red neuronal?

Si los pesos se inicializan mal (muy grandes, muy pequeños, o todos iguales), las activaciones y los gradientes pueden explotar o desvanecerse desde las primeras iteraciones, impidiendo que la red aprenda. Una buena inicialización pone los pesos en un rango que mantiene los gradientes en magnitudes razonables a lo largo de toda la red.

¿Qué podría ocurrir si todos los pesos se inicializan con el mismo valor?

Todas las neuronas de una capa computarían exactamente la misma salida y recibirían exactamente el mismo gradiente, por lo que todas se actualizarían de la misma manera. La red nunca rompería esta simetría y todas las neuronas aprenderían la misma función, lo cual equivale a tener una sola neurona por capa. Es el problema de *symmetry breaking*.

¿Cuál es la diferencia entre las inicializaciones de Xavier (Glorot) y He?

Ambas inicializan los pesos de forma aleatoria pero escalan la varianza según el número de conexiones de entrada y salida. Xavier fue diseñada para activaciones simétricas como Tanh y Sigmoid, y escala la varianza como $2 / (\text{fan_in} + \text{fan_out})$. He (Kaiming) fue diseñada para ReLU, que descarta la mitad de las activaciones (las negativas), y usa $2 / \text{fan_in}$ para compensar esa reducción de varianza.

¿Por qué en una red con ReLU suele usarse la inicialización de He?

ReLU pone a cero todas las activaciones negativas, lo que reduce la varianza de la salida aproximadamente a la mitad respecto a la entrada. Xavier no tiene en cuenta esto y deja las varianzas más pequeñas de lo deseable. He corrige este factor multiplicando la varianza por 2, manteniendo la señal en magnitudes razonables a través de las capas.

¿Qué capas de una red requieren inicialización explícita y cuáles no?

Las capas `nn.Linear` y `nn.Conv2d` tienen pesos que deben inicializarse bien (por defecto PyTorch usa Kaiming uniforme, que es razonable). Las capas `nn.BatchNorm` inicializan $\gamma=1$ y $\beta=0$ por defecto, lo cual es correcto y no suele necesitar modificación. Las capas de activación (ReLU, Dropout) no tienen parámetros aprendibles.

Preguntas prácticas (resultados esperados al ejecutar)

¿Qué diferencias notaste en la convergencia del modelo según la inicialización?

He (Kaiming) suele converger más rápido y de forma más estable con ReLU. Xavier funciona aceptablemente. La inicialización uniforme aleatoria sin escalar puede causar gradientes muy grandes o muy pequeños en las primeras iteraciones.

¿Alguna inicialización provocó inestabilidad (pérdida muy alta o NaNs)?

La inicialización uniforme con rangos grandes (por ejemplo `uniform_(-1, 1)` en una red profunda) puede provocar explosión de gradientes y pérdidas muy altas o NaN, especialmente sin BatchNorm.

¿Qué impacto tiene la inicialización sobre las métricas de validación?

Una buena inicialización puede alcanzar mejores métricas de validación en menos épocas. Con mala inicialización la red puede quedar atrapada en mínimos subóptimos.

¿Por qué bias se suele inicializar en cero?

A diferencia de los pesos, los bias no necesitan romper simetría (no conectan neuronas entre sí). Inicializarlos en cero es una convención segura que no introduce ventaja ni desventaja para ninguna neurona en particular.